面试自我介绍 | 技术经理

# haswhere

## B2B 大宗商品在线交易平台 · 技术经理

13 年一线技术经验 | 20 人研发团队 | 异构架构治理 C + Java

业务 × 技术双能力 | B2B 交易平台从 0 到 N | AI 工程化落地

186-****-3592 | 815841365@qq.com | 郑州

# 13 年一线技术，专注大宗商品交易这一个领域

现任 B2B 大宗商品在线交易平台技术经理 · 带 20 人研发团队，向总经理汇报

## 个人简介

13 年一线技术经验，现任 B2B 大宗商品在线交易平台技术经理，负责业务中台（12+ 微服务）与数据服务中心的架构设计。平台服务大额资金交易场景——一条链路串起交易、支付、仓单、税务多个系统，任何环节出问题都是真金白银的风险。

- **驾驭异构架构：**自研 DS 数据中心（防腐层）管住 C 核心引擎与 Java 微服务的全链路通信
- **技术 × 业务交叉：**把客户一句话翻译成技术方案，守住「不让定制需求污染主链路」的红线
- **用机制替代亲力亲为：** On-call 轮值 / 技术债台账 / 接口契约治理三件套
- **至今保持在一线：**方案不飘——坑在哪、工作量多少，我能直接判断

EXPERIENCE
**13 年一线**
专注大宗商品交易与资金场景

TEAM
**20 人团队**
3 C / 6 Java / 5 Vue / 5 测试

ARCHITECTURE
**12+ 微服务**
业务中台 + DS 数据中心两层

ROLE
**技术经理**
Java 侧负责人 · 向总经理汇报

# 工作经历：从清算开发到 20 人团队技术经理

同一领域深耕 13 年——前 2 年金融级清算打底，后 11 年在同一业务域做到平台化； 2024 起延伸 AI 支线

### 2013 — 2015

**Java 开发工程师**

某软件公司 · 清算 / GIS

- 清算与 GIS 后端开发
- 日处理交易量超 10 万笔
- 数据精确性 > 一切

### 2015 — 2020

**核心开发工程师**

某软件公司 · 交易系统

- 从 0 到 1 参与交易系统建设
- 参数化配置、多模式交易
- 外部系统对接

### 2020 — 至今

**技术经理（Java 侧负责人）**

某软件公司 · B2B 交易平台

- 带 20 人研发团队
- 平台化 + 配置化 + 异构治理
- 建规范体系与管理三件套

### 2024 — 至今

**AI 前期技术原型**

同一业务域的技术栈延伸

- 模型选型 + 私有化验证
- 多格式文档落向量库
- 跑通 RAG 地基

**13 年 = 2 年清算 + 11 年本司； 20 人 = 3 C / 6 Java / 5 Vue / 5 测试 —— 这两个数字必须脱口而出，全篇一致。**

# 先给结果，再讲过程

运行时真实口径；涉及考核线的数字一律带「当时设定的团队目标」限定语

RELIABILITY

**3 年 +**

零资金差错 · 冬储高峰连续两季零 P0

DELIVERY

**2 月 → 1 周**

新客户上线周期（配置化改造）

INTEGRATION

**3 周 → 5 天**

外部系统对接周期（统一适配层）

AVAILABILITY

**99.95%**

外部接口平均可用率

EFFICIENCY

**5 天 → 1.5 天**

一笔交易周期

RECONCILIATION

**9 人天 → 2 小时**

财务对账（三流自动勾稽）

TEAM

**45% → 15%**

团队被动响应占比

AUTONOMY

**80%**

线上问题团队独立处理

**不要逐个念——挑三个讲：三年零资金差错、 2 个月 → 1 周的上线提速、被动响应 45% → 15% 。**

# 产品定位：把「签了合同之后的事」全部线上化、可信化、金融化

大宗商品 B2B 在线交易平台 · 技术经理 / Java 侧负责人 · 覆盖交易 · 支付结算 · 履约 · 仓单 · 物流 · 税务 · 供应链金融

WHAT · 是什么

**大宗商品 B2B 在线交易平台**

MODES · 交易模式

**挂牌 / 竞价 / 询价 / 招标 / 点价**

WHO · 服务谁

**能源化工国企 • 贸易商 • 产业买方**

VALUE · 价值

**发现价格 • 阳光采购 • 供应链金融**

死结 01 · 信任

## 跟陌生人做生意不敢

大额交易、决策链长；先打款怕不给货、先发货怕不结款，对「信任」的敏感度远高于对「价格」。

解法：平台担保 + 资金监管，钱不过对方手；履约率与成交记录公开。

死结 02 · 效率

## 一笔交易跑断腿

比价靠打电话、合同来回寄、催货靠喊、对账靠翻凭证——一笔交易 20 个电话。

解法：询价 → 合同 → 支付 → 仓储 → 物流 → 质检 → 结算 → 开票全链路线上化。

死结 03 · 资金

## 有好机会但缺钱

货压仓库占资金、账期 30–90 天拖垮现金流，有好机会也吃不下。

解法：预付款融资 / 仓单质押 / 应收账款融资——让钱不够的买方也能成交。

**价值主张：让大额交易从「靠人脉、靠电话」变成「靠数据、靠系统、靠信用」——技术在这里不是成本，是增长引擎。**

# 四层架构：接入层 → midservice 网关 → DS 数据中心 → 业务中台

不写 C 代码，但通过 DS 防腐层管住 C 核心引擎与 Java 微服务的全链路通信

| | | | |
|---|---|---|---|
| **接入层 三端** | **PC 端 / 机构端**<br>Vue3 全栈 | **移动端**<br>触屏适配 | **外部生态 9+**<br>银行 / 物流 / 税务 / 签章 / 供应链金融 |
| **midservice 网关** | **鉴权 · 限流**<br>统一入口安全控制 | **路由 · 分发**<br>承载 12+ 业务服务 | **接口聚合**<br>对外 API 统一出口 |
| **DS 数据中心** | **协议转换**<br>TCP 自定义二进制 ↔ Hessian 标准接口 | **可靠性**<br>重试 / 幂等 / 顺序保证 | **可观测性**<br>链路追踪 · 指标 · 告警收敛 |
| **业务中台** | **12+ 业务服务**<br>交易模式 / 履约保证 / 规则配置 / 生态对接 | **midmon 监控网关**<br>指标 · 健康度 · 告警 | **C 核心引擎**<br>内存撮合 · 多模式交易状态机 · 确定性优先 |

**C 核心引擎换确定性 · DS 防腐层换边界可控——异构复杂度收敛在一层，两侧独立演进、互不阻塞。**

# 三个组件不是堆技术，是三次取舍

每个选择背后都写清了「换到什么」和「付出什么」

C · 核心引擎

## 换确定性

**换到：**

内存撮合、毫秒级稳定延迟，不受 GC 停顿影响

**付出：**

迭代慢、 C 人才稀缺、与 Java 生态割裂

决策：否决重写，保留 C 兜住资金链路

Java · 业务中台

## 换迭代速度

**换到：**

12+ 微服务快速迭代，生态与人才充足

**付出：**

性能天花板、依赖治理成本

决策： 12+ 微服务承载业务迭代与定制

DS · 数据中心

## 换边界可控

**换到：**

异构边界收敛在一层，可观测、可治理

**付出：**

多一层转发、多一套需要维护的组件

决策：协议 / 可靠性 / 可观测收敛一层

**我拍板的关键决策：否决「 C 核心引擎重写成 Java 完成微服务化」——撮合是资金链路心脏， GC 停顿不可接受。**

# 我带队解决的问题——每个都有打法、有结果

不是「遇到就修」，是先归因到架构层，再用机制兜住

01 · 资金一致性

## 跨月长事务

**打法：**

Saga + 状态机 + 事件溯源

**结果： 3 年零资金差错**

02 · 架构红线

## 定制污染主链路

**打法：**

多租户配置化（策略模式 + 配置模板）

**结果：上线 2 月 → 1 周，代码零膨胀**

03 · 异构通信

## C ↔ Java 边界失控

**打法：**

DS 防腐层收敛协议 / 可靠性 / 可观测

**结果：两团队独立演进，互不阻塞**

04 · 系统集成

## 外部对接慢

**打法：**

统一网关适配层，覆盖 9+ 外部系统

**结果： 2-3 周 → 3-5 天，可用率 99.95%**

05 · AI 协同提效

## 推动 AI 协同开发

**打法：**

把 AI 协同开发引入研发流程，提效的同时沉淀团队可复用资产 SKILLS

**结果：能力沉淀为可复用资产，团队整体提效**

06 · 数据同步

## 跨模块信息丢失

**打法：**

Kafka 事件驱动 + 本地消息表 + 幂等 + 对账兜底

**结果：根治，沉淀为契约治理规范**

**问题先归因，再定打法，最后拿结果验收——这些打法我换个场景也能直接用。**

# 我把研发过程做成「可预测、可追踪、可复盘」

流程规范 + 工程规范 + 管理机制 —— 关键节点强制过审，用机制替代亲力亲为

**需求收口**
统一入口 · 评审

→

**设计评审**
方案 · 架构过审

→

**开发**
强制代码评审

→

**QA 验收**
用例 · 准出

→

**发布评审**
上线前闸门

→

**系统移交**
文档 · 交接

PROCESS · 流程规范

## 全流程闸门

覆盖需求、设计、开发、测试、发布、移交六阶段，关键节点强制过审；配套年度技术建设规划与故障复盘闭环，根因回灌规范。

ENGINEERING · 工程规范

## 守住架构红线

接口契约治理（管连线不管黑盒）；主链路强制代码评审；沉淀原型库与开发 skill，新人照着 skill 可独立交付典型功能。

MECHANISM · 管理机制

## 机制替代人治

On-call 轮值（主副双岗 + 四级 SLA）；技术债台账（20% 带宽固定偿还）；师徒制 + owner+backup 双人制，四维绩效考核。

**规范的目的不是加流程，是让研发可预测、可追踪、可复盘——把个人能力沉淀成团队能力。**

# 我带队拿到的四本账

交付 · 质量 · 效率 · 能力——不是个人成绩单，是体系结果

01 · DELIVERY

## 交付

- 12+ 微服务业务中台，从 0 到 1 建成
- 销售端 + 采购端双平台并线交付
- 9+ 外部系统对接（银行 / 仓储 / 税务 / 监管）

02 · QUALITY

## 质量

- 3 年 + 零资金差错（交易 / 结算 / 清算）
- 外联接口平均可用率 99.95%

03 · EFFICIENCY

## 效率

- 新客户上线 2 月 → 1 周
- 系统对接 2-3 周 → 3-5 天
- 交易周期 5 天 → 1.5 天，对账 9 人天 → 2 小时

04 · CAPABILITY

## 能力

- 20 人四梯队，核心模块无单点依赖
- 团队独立处理 80% 线上问题，被动响应 45% → 15%
- 沉淀规范 / 排查手册 / 原型库 / 开发 skill

**追求的不是短期救火，而是长期可复制的研发体系——交付有节奏、质量有底线、效率可量化、能力能传承。**

# 两个客户、两种业态，验证同一套平台底座

民营贸易商（广西晔龙）· 国有要素交易场所（广西林控）——均为我方承建

EELONG · SHARE

**>15%**

进口菜粕全国第一 · 年贸易量 50 万吨

EELONG · CUSTOMER

**3000 家**

下游饲料企业 · 郑商所产融基地

LINKONG · BIDDING

**32 轮**

单场竞价轮次 · 溢价 18.2%

LINKONG · MARKET

**2022**

国有全资平台上线 · 面向东盟

CASE 01 · 广西晔龙国际贸易

## 民营贸易商：期现结合

画像：全国最大进口菜籽粕贸易商（份额 >15%），年贸易量 50 万吨，下游客户约 3000 家。

我们解决：套保策略绑定现货合同、敞口实时计算、风险限额分级预警；预售与远期采购的履约批次拆分。

**价值：把「拍脑袋锁价」变成可量化、可预警的期现风控。**

CASE 02 · 中国 - 东盟林业资源平台

## 国有要素场所：非标资产线上化

画像：广西林业局批准的国有全资平台（2022 上线），品种覆盖活立木、林地流转、原木、人造板。

我们解决：活立木等非标资产线上化——标的清单结构化、延时竞价以服务器时间为基准、保证金三层配置。

**价值：公开竞价 + 全程留痕 + 结果公示，守住国有资产不被「贱卖」。**

**一个是民营贸易商的「活下来、赚价差」，一个是国企的「不贱卖、说得清」——同一套底座，两种价值主张。**

# 各类型文档 → 向量数据库：我把 RAG 的地基打扎实

2024 年至今 · B2B 主线之外，做 AI 智能服务平台的前期技术原型：模型选型验证 + 多格式文档落向量库

01
**多格式文档**
PDF / Word / Excel / 网页 / 图片

→

02
**解析与清洗**
结构化抽取 · 去噪

→

03
**切分**
Chunking：按语义 / 标题层级

→

04
**向量化**
Embedding：文本 → 向量

→

05
**向量数据库**
入库索引 · 相似度检索

前期技术原型

## 选型 + 私有化验证

模型选型与内网私有化（vLLM）可行性验证，跑通「文档 → 向量库 → 检索」端到端链路。

通道判断

## 数据敏感度决定通道

通用咨询走第三方大模型快速见效；涉密数据走内网私有化，合规与成本都占住。

**AI 是主线之外的一块拼图——我把重心放在前期原型与文档落库，这一步决定 RAG 能不能用、检索准不准。**

# 三个标签，钉住记忆

业务翻译官 · 技术操盘手 · 流程守护者

01 · POSITIONING

## 业务翻译官

把「支持分批付款」五个字拆成 14 个子需求、识别 3 个风险点。

能在评审中反问：这个合同条款的违约触发条件，系统哪个状态节点对应？

落地动作：需求评审先翻译，再谈方案

02 · CAPABILITY

## 技术操盘手

在 C + Java 异构架构里做选型、评审、技术债管理。

不写 C 代码，但通过防腐层管住异构通信全链路。

落地动作：异构边界明确，方案先讲坑

03 · MECHANISM

## 流程守护者

建立 On-call 轮值、技术债台账、接口契约治理三大机制。

让团队从被动救火转向主动建设。

落地动作：被动响应 45% → 15%

**责任金句：技术经理的价值不是方案多漂亮，是对可实施性与落地效果负责——坑在哪、工作量多少、风险在哪一层，我能直接判断。**

# 我的落地打法：先止血、再还债、后增值

接手一个团队或系统，我先用机制把确定性拉起来，再用复用与 AI 提效

第一阶段

## 先止血

- 稳线上：盯核心链路与资金相关告警
- 补齐监控 / 告警 / On-call 与故障流程
- 梳理主链路，识别单点与最大风险

第二阶段

## 再还债

- 建技术债台账， 20% 带宽固定偿还
- 推接口契约治理，收敛跨模块耦合
- 补关键回归与自动化，减少手工发布

第三阶段

## 后增值

- 复用底座与配置化，压缩新需求交付周期
- AI 提效：知识检索 / 研发辅助落地
- 沉淀规范与人才梯队，机制可自转

**先让团队从救火里抽身，再用复用和 AI 把效率拉起来——这是我换了场景也能复用的打法。**

SUMMARY · 三个标签

# 业务翻译官 · 技术操盘手 · 流程守护者

> **责任金句：**技术经理的价值不是方案多漂亮，是对可实施性与落地效果负责——坑在哪、工作量多少、风险在哪一层，我能直接判断，不给团队挖坑。

THANK YOU

# 我的介绍完了。谢谢